

Capítulo 13

Quantization

Construindo um Modelo de Linguagem do Zero

Curso bilingue inspirado no LLM101n

Gerado em 10/08/2026

Capítulo 13 — Inference II: Quantization

Objetivo de aprendizagem: encolher o modelo trocando `float32` por inteiros de 8, 4 ou menos bits — implementando a aritmética do zero — e medir as **três** coisas que importam: tamanho, qualidade e velocidade. Uma delas não vai melhorar, e entender por que é metade do capítulo.

Pré-requisitos: Capítulos 9 (precisão), 11 (o modelo treinado) e 12 (por que o decode é limitado por memória).

O checkpoint do Capítulo 11 está **versionado** — tudo aqui roda logo após o clone.

Arquivos:

- `quantizacao.py` — a aritmética do zero: escala, zero-point, per-channel
- `quantizar_modelo.py` — aplica ao modelo do Cap. 11 e mede
- `exercicios.md` — exercícios

1. Por que encolher os pesos

O Capítulo 12 estabeleceu que a fase de **decode** é limitada por **memória**: para produzir *um* token é preciso ler *todos* os pesos do modelo. O cálculo é pequeno; o tráfego de memória é tudo.

Quando o gargalo é ler bytes, a intervenção óbvia é ter menos bytes para ler.

Formato	Bytes por peso	Um 7B ocupa
<code>float32</code>	4	28 GB
<code>bfloat16</code>	2	14 GB
<code>int8</code>	1	7 GB
<code>int4</code>	0,5	3,5 GB

A diferença entre 14 GB e 3,5 GB é a diferença entre "precisa de uma placa de datacenter" e "roda no seu notebook". É por isso que quantização deixou de ser assunto de especialista.

2. A aritmética, que é mais simples do que parece

Quantizar é achar uma **escala** que mapeie o intervalo dos seus floats no intervalo dos inteiros, e depois arredondar.

```
qmax = 127 # int8 vai de -128 a 127
escala = x.abs().max() / qmax
```

```
q = torch.round(x / escala) # agora cabe em 8 bits
```

E voltar é uma multiplicação:

```
x_recuperado = q * escala
```

Rodando em um tensor de verdade:

```
original : +0.1808 -0.0700 -0.3596 -0.9152 +0.6258
int8      : 11 -4 -21 -54 +37
de volta  : +0.1871 -0.0680 -0.3572 -0.9184 +0.6293

escala = max|x| / 127 = 0,017008
erro relativo: 0,49%
```

Meio por cento de erro, e o tensor ocupa **um quarto** do espaço. É esse o negócio que a quantização oferece.

3. Simétrica ou assimétrica — e uma armadilha que me pegou

A versão acima é **simétrica**: uma escala só, e o zero do float é o zero do inteiro. Funciona bem quando os dados são centrados em zero.

A **assimétrica** acrescenta um *zero-point*, o inteiro que representa o float **0.0**:

```
escala = (xmax - xmin) / (qmax - qmin)
zero = round(qmin - xmin / escala)
q = round(x / escala + zero)
```

Ela aproveita o intervalo inteiro quando os dados não são centrados. Medindo:

Distribuição	Simétrica	Assimétrica
normal (centrada em 0)	0,782%	0,724%
só positiva (pós-ReLU)	0,595%	0,277%
deslocada (média 5)	0,358%	0,177%

Pesos de rede costumam ser centrados em zero — a simétrica basta, e sai mais barata (não há zero-point para somar em cada operação). **Ativações**, depois de ReLU ou GELU, não são; aí a assimétrica ganha o dobro.

A armadilha: o zero-point pode não caber

A primeira versão deste capítulo media, para a distribuição de média 5, **4,16%** de erro na assimétrica contra 0,36% na simétrica. Doze vezes pior, exatamente onde ela deveria ganhar.

A causa, encontrada imprimindo os números intermediários:

```
dados em [1,24 ; 8,63], escala = 0,02898
zero-point IDEAL = -170,7
intervalo de int8 = [-128, 127] -> não cabe
```

Quando o intervalo dos dados **não contém o zero**, o zero-point necessário cai fora do que o int8 representa. O `clamp` o força a -128, o mapeamento inteiro se desloca, e os valores grandes saturam no teto.

A correção é padrão — PyTorch, TFLite e ONNX fazem o mesmo — e consiste em **estender o intervalo para incluir o zero**:

```
xmin = min(xmin, 0)
xmax = max(xmax, 0)
```

Custa um pouco de resolução (parte do intervalo passa a representar valores que não ocorrem) e garante duas coisas: o zero-point é representável, e o float `0.0` tem um inteiro exato — o que importa para *padding* e para máscaras.

4. Per-tensor vs per-channel, e uma métrica que mente

Até aqui usamos **uma** escala para a matriz inteira. A alternativa é uma escala **por linha** (por canal de saída).

Numa matriz 8×64 onde uma linha tem valores 100x maiores que as outras:

Métrica	Per-tensor	Per-channel
erro na matriz inteira	1,32%	0,50%
erro nas 7 linhas normais	48,43%	0,58%
erro na linha grande	0,50%	0,50%

Olhe a primeira linha da tabela e depois a segunda. Pelo erro global, o per-tensor parece aceitável — 1,32%. Ele não é: as sete linhas normais estão com **48% de erro**, praticamente destruídas.

A métrica global mentiu porque é normalizada pela norma da matriz, e essa norma é dominada pela linha grande. O erro está todo nas linhas pequenas, que quase não contribuem para o denominador.

A lição vale além da quantização. Um erro médio baixo pode esconder um subconjunto completamente arruinado. Sempre que houver heterogeneidade de escala nos dados, desagregue a métrica antes de concluir.

O mecanismo: uma escala única é refém do **maior** valor da matriz. A linha 100x maior define a escala, e as outras sete passam a caber em pouquíssimos níveis inteiros — perto de zero, viram zero.

O per-channel custa **28 bytes** neste exemplo e resolve. É por isso que praticamente toda quantização de peso na prática é per-channel.

5. Quantos bits o modelo aguenta?

Erro de reconstrução do peso **não é** erro do modelo. A única forma de saber é medir a loss. Aplicando ao modelo do Capítulo 11:

Bits	Loss de validação	Piora	Perplexidade
float32	3,9372	—	51,3
int8	3,9371	-0,0001	51,3
int6	3,9393	+0,0021	51,4
int4	4,0008	+0,0636	54,6
int3	4,4029	+0,4657	81,7
int2	7,4670	+3,5298	1.749

int8 é de graça. A loss não piorou — variou na quarta casa decimal, dentro do ruído. Quatro vezes menos memória, zero de custo em qualidade.

int4 custa 0,06 de loss e ainda é perfeitamente utilizável. É por isso que modelos de 4 bits são hoje o padrão para rodar LLM em hardware modesto.

Entre 4 e 3 bits está o precipício. A piora salta de 0,06 para 0,47 — quase oito vezes. E int2 destrói o modelo: perplexidade 1.749 contra 51.

Note que o erro de *reconstrução do peso* cresce suavemente (0,69% -> 2,84% -> 12,6% -> 29,3%), enquanto a *loss do modelo* tem um joelho abrupto. Um não prevê o outro. Meça o que você quer saber.

6. Quanto o modelo encolhe

Onde estão os parâmetros	Quantidade	Fração
camadas Linear	1.966.080	90%
embeddings e LayerNorm	230.272	10%

Configuração	Tamanho	Redução
--------------	---------	---------

tudo em <code>float32</code>	8,79 MB	—
<code>Linear</code> em <code>int8</code>	2,89 MB	3,04x
<code>Linear</code> em <code>int4</code>	1,90 MB	4,61x
tudo em <code>int8</code>	2,20 MB	4,00x

Quantizar só as camadas `Linear` dá **3,04x**, não 4x — e a conta fecha exatamente: $0,9/4 + 0,1 = 0,325$, ou seja 3,08x. Os 10% que ficam em `float32` explicam toda a diferença.

(!) **A linha do `int4` é teórica.** Ela supõe **empacotamento**: dois pesos por byte. A implementação deste capítulo não empacota — guarda cada valor num `int8` mesmo usando só 4 bits de resolução. Empacotar é trabalho de serialização, não de quantização, e formatos como o GGUF fazem isso. O número está na tabela como promessa, não como medição, e vale saber qual é qual.

7. E a velocidade? Piorou.

Configuração	Gerar 64 tokens
<code>float32</code>	0,126 s
<code>int8</code>	0,174 s (0,72x)

O modelo quantizado ficou mais lento, e não é bug.

Esta implementação guarda o peso em `int8` e o **reconstrói em `float32`** a cada forward. Ou seja: ela paga a desquantização e depois faz exatamente a mesma `matmul` de antes. A economia de memória é real; a de velocidade não existe, porque a conta não mudou.

Para a quantização acelerar de verdade é preciso um **kernel** que multiplique `int8 × int8` acumulando em `int32`, sem passar por float. Isso não se escreve em PyTorch puro — é o que llama.cpp, bitsandbytes e o ONNX Runtime fornecem.

"O modelo ficou 4x menor" e "o modelo ficou mais rápido" são afirmações independentes. A primeira decorre da representação. A segunda exige que alguém tenha escrito o kernel.

É a terceira vez que o curso encontra a mesma forma de erro: no [Capítulo 8](#) a GPU era mais lenta para modelo pequeno, no [Capítulo 12](#) o KV-cache rendeu 1,9x em vez de 60x, e aqui a quantização deixa mais lento. **Otimização é uma hipótese até ser medida.**

8. O que a prática faz além disto

O que construímos é **round-to-nearest** (RTN) per-channel: a linha de base honesta. Os métodos usados em produção atacam duas limitações dela.

Escolher melhor o arredondamento. RTN arredonda cada peso isoladamente. **GPTQ** e **AWQ** usam um conjunto pequeno de dados de calibração para escolher os arredondamentos que menos alteram a **saída** da camada — não o peso. É a diferença entre minimizar o erro no que você guarda e no que você produz.

Lidar com outliers de ativação. Quantizar pesos é fácil; quantizar **ativações** é difícil, porque em Transformers algumas dimensões têm valores muito maiores que as demais — e você já viu, na Seção 4, o que um outlier faz com uma escala compartilhada. **LLM.int8()** resolve separando essas dimensões e calculando-as em `float16`; **SmoothQuant** redistribui a magnitude entre ativação e peso antes de quantizar.

Os dois problemas são a Seção 4 deste capítulo em escala real.

Quantização por grupos: o meio-termo que todo mundo usa

Entre o per-tensor (1 escala) e o per-channel (uma por linha) existe o esquema adotado por GGUF e GPTQ: **uma escala a cada G valores consecutivos** da linha, tipicamente $G = 64$ ou 128 .

Numa matriz 256×256 onde 32 colunas têm valores 50x maiores, tudo em `int4`:

Esquema	Erro global	Erro nas colunas normais	bits/peso reais
per-channel	11,23%	100,00%	4,06
grupos de 128	10,51%	65,90%	4,12
grupos de 64	10,13%	38,57%	4,25
grupos de 32	9,93%	9,69%	4,50

Repare que o **erro global quase não muda** — e as colunas normais, que são 87,5% da matriz, saem de *completamente apagadas* para 9,7% de erro. É a Seção 4 outra vez: a métrica agregada esconde exatamente o efeito que interessa.

E há um custo que raramente aparece nas discussões:

```
bits_reais = bits + (bits_da_escala / G)
```

Com `int4` e $G = 64$, guardando a escala em `fp16`, cada peso ocupa **4,25 bits** — não 4. Com $G = 32$, **4,5 bits**. É esse compromisso que os nomes dos formatos GGUF codificam: `Q4_0`, `Q4_K_M` e afins diferem no tamanho do grupo e em quantos bits gastam com metadados.

9. Resumo do capítulo

- Quantizar é achar uma **escala**, dividir e arredondar. A aritmética cabe em cinco linhas.

- **Simétrica** para pesos (centrados em zero), **assimétrica** para ativações — e o intervalo precisa **conter o zero**, ou o zero-point não cabe no inteiro.
- **Per-channel**, sempre. E cuidado com métricas globais: elas escondem subconjuntos destruídos.
- **int8 é de graça** neste modelo (-0,0001 de loss). **int4 custa 0,06**. Entre 4 e 3 bits há um precipício.
- Erro no peso **não prevê** erro no modelo. Meça a loss.
- **Encolher não é acelerar**. Sem kernel inteiro, a quantização economiza memória e custa tempo.

Próximo capítulo

Capítulo 14 — Finetuning I: SFT. Até aqui o modelo só sabe continuar texto. O *supervised finetuning* é o que transforma um continuador de texto em algo que responde ao que você pede.

Exercícios — Capítulo 13 (Quantização)

Faça na ordem. Soluções comentadas em [solucoes/](#) — tente antes de olhar.

Estes exercícios usam o checkpoint do Capítulo 11 ([../11-datasets/modelo.pt](#)).

E1 — A aritmética (aquecimento)

Sem rodar:

1. Um tensor tem valores em `[-3, 3]`. Qual a escala simétrica para int8? Qual inteiro representa o float `1,5`? E que float se recupera dele?
 2. Por que a quantização simétrica usa `[-127, 127]` em vez de `[-128, 127]`, se o int8 comporta o -128?
 3. Um tensor tem todos os valores em `[10, 11]`. Qual o erro relativo esperado com quantização **simétrica** de 8 bits? E por que a assimétrica ajudaria tanto aqui?
-

E2 — Reproduza a armadilha do zero-point (importante)

No `quantizacao.py`, remova as duas linhas que estendem o intervalo para incluir o zero (`xmin = min(xmin, 0)` e `xmax = max(xmax, 0)`).

1. Meça o erro da assimétrica para uma distribuição `randn(1000) + 5`. Compare com a simétrica. Qual ganha?
 2. Imprima o *zero-point ideal* (`qmin - xmin/escala`) antes do `clamp`. Ele cabe em int8?
 3. Quantos valores saturam no teto (`q == 127`)? Explique a cadeia causal completa: do zero-point fora do intervalo até o erro medido.
-

E3 — A métrica que mente (importante)

Reproduza a matriz 8×64 com uma linha 100x maior.

1. Meça o erro **global** e o erro **só nas 7 linhas normais**, com per-tensor. Por que os dois números são tão diferentes?
 2. A norma usada em `erro_relativo` é dominada por quê? Escreva a conta.
 3. Proponha **outra** métrica que teria denunciado o problema sem precisar desagregar por linha. (Dica: pense em erro relativo *por elemento*.)
-

E4 — Onde fica o precipício

Rode o `quantizar_modelo.py`.

1. Faça a tabela de duas colunas: *erro de reconstrução do peso* (do `quantizacao.py`) e *piora da loss* (do `quantizar_modelo.py`), para 8, 6, 4, 3 e 2 bits. Uma prevê a outra?
2. Entre quais bits está o joelho? O que acontece com a perplexidade ali?
3. Se você tivesse que escolher um número de bits para servir este modelo, qual escolheria? Justifique com os dois eixos (qualidade e tamanho), não com um só.

E5 — Quantize os embeddings também

O `quantizar_modelo.py` tem o argumento `incluir_embeddings`.

1. Ative-o e meça: quanto o modelo encolhe a mais? Quanto a loss piora?
2. Neste modelo os embeddings são 10% dos parâmetros. Num modelo de 7B são ~2%. Em qual dos dois vale mais a pena quantizá-los, e por quê a resposta **não** é "no que tem a maior fração"?
3. A camada de saída (`lm`) mapeia `n_embd -> vocab_size`. Ela é um `nn.Linear`, então já está sendo quantizada. Há algum motivo para tratá-la com mais cuidado que as outras?

E6 — Meça o que a memória realmente faz

1. Meça o tempo de gerar 64 tokens com `float32` e com `int8`. Qual é mais rápido?
2. Explique, a partir do `forward` da `LinearQuantizada`, exatamente onde o tempo extra é gasto.
3. Escreva o que precisaria mudar para que a quantização de fato acelerasse. Por que isso não se faz em PyTorch puro?

E7 — Quantização por grupos (desafio)

Entre o per-tensor (1 escala) e o per-channel (uma por linha) existe o meio-termo usado por GGUF e GPTQ: uma escala a cada **G** valores consecutivos da linha (tipicamente $G = 64$ ou 128).

1. Implemente. Para uma matriz (`saida, entrada`), isso significa `entrada/G` escalas por linha.
2. Meça o erro de reconstrução com $G = 256$ (equivalente ao per-channel de uma linha de 256), 128, 64 e 32. Quanto se ganha?
3. Faça a conta do **custo real em bits por peso**: com `int4` e $G = 64$, quantos bits cada peso ocupa de fato, contando as escalas em `float16`? Compare com os 4 bits nominais.